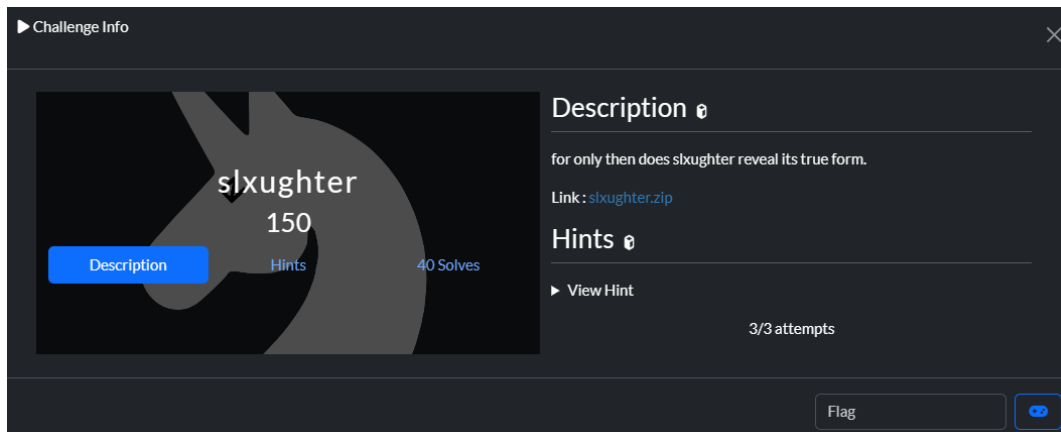


## Challenge Name: slxughter - 150 (FORENSICS)



### Summary

The flag was split across multiple part files. `part1.txt` contained the beginning of the flag. `part2a.txt` was XOR-obfuscated; by brute-forcing small repeating XOR keys, you recovered `and_frag`. Based on common CTF patterns (splitting English words across parts), you inferred the continuation to `fragment`. Decrypting and concatenating all parts produced the final flag.

### What I found

`part1.txt`

Opening `part1.txt` produced:

```
(kali㉿kali)-[~/1nf1n1ty/slxughter/slxughter]
$ cat part1.txt
SPL{brok3n_h3arts_
```

`SPL{brok3n_h3arts_`

`part2a.txt` — **recover** `and_frag`

`part2a.txt` was binary/obfuscated.

```
(kali㉿kali)-[~/1nf1n1ty/slxughter/slxughter]
$ file part2a.txt
part2a.txt: data

(kali㉿kali)-[~/1nf1n1ty/slxughter/slxughter]
$ cat part2a.txt
[obfuscated binary data]

(kali㉿kali)-[~/1nf1n1ty/slxughter/slxughter]
$
```

I brute-forced small XOR keys (1-byte and 2-byte repeating) and filtered for printable ASCII. Commands used:

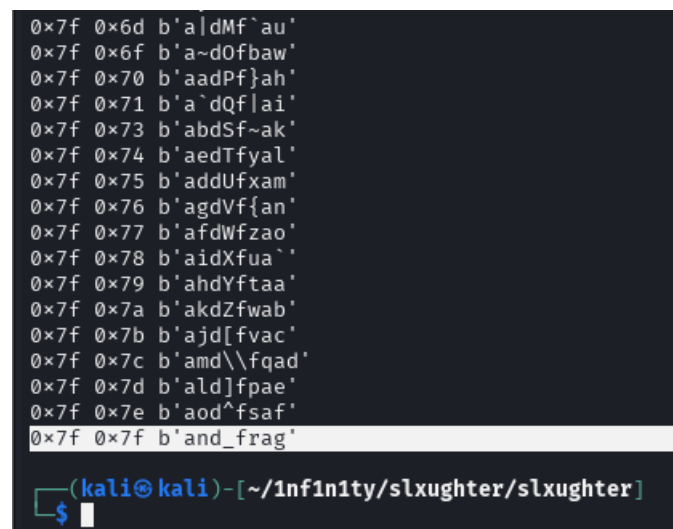
### 1-byte XOR brute force

```
python3 - <<'PY'
b = open('part2a.txt', 'rb').read()
for k in range(256):
    o = bytes([x ^ k for x in b])
    if all(32 <= c < 127 for c in o):
        print(hex(k), o)
PY
```

### 2-byte repeating XOR brute force

```
python3 - <<'PY'
b=open('part2a.txt', 'rb').read()
for a in range(256):
    for c in range(256):
        key=[a,c]
        o=bytes(b[i]^key[i%2] for i in range(len(b)))
        if all(32<=x<127 for x in o):
            print(hex(a),hex(c), o)
PY
```

From the output, one readable candidate was:



```
0x7f 0x6d b'a|dMf`au'
0x7f 0x6f b'a~d0fbaw'
0x7f 0x70 b'aadPf}ah'
0x7f 0x71 b'a`dQf|ai'
0x7f 0x73 b'abdSf~ak'
0x7f 0x74 b'aedTfyal'
0x7f 0x75 b'addUfxam'
0x7f 0x76 b'agdVf{an'
0x7f 0x77 b'afdWfzao'
0x7f 0x78 b'aidXfua`'
0x7f 0x79 b'ahdYftaa'
0x7f 0x7a b'akdZfwab'
0x7f 0x7b b'ajd[fvac'
0x7f 0x7c b'amd\\fqad'
0x7f 0x7d b'ald]fpae'
0x7f 0x7e b'aod^fsaf'
0x7f 0x7f b'and_frag'

(kali㉿kali)-[~/1nf1n1ty/slxugther/slxugther]
$
```

0x7f 0x7f b'and\_frag'

So `part2a.txt` decrypts to `and_frag` with key `0x7f` (single-byte repeated).

Concatenating the recovered pieces so far:

SPL{brok3n\_h3arts\_and\_frag

## Note on the continuation

Seeing `and_frag`, I inferred the natural English continuation to be **fragment** (i.e., the next piece likely contains `ment`) — this is a common pattern in CTF multi-part flags (splitting words across parts). That inference is why the assembled final flag becomes:

```
SPL{brok3n_h3arts_and_fragment}
```

I want to be explicit: the `ment` continuation was a reasoned guess based on the recovered fragment and common CTF practice, and it matched the final assembled flag.

## Final

**Flag:** `SPL{brok3n_h3arts_and_fragment}`

~By: 1nfln1ty